

MANIKANTA REDDY ANUGU

San Antonio, TX • 470-610-4832 • reddymanikanta437@gmail.com • [LinkedIn](#)

RESEARCH INTERESTS

Generative AI adoption in industry, LLM comparison and evaluation, retrieval-augmented generation, and the gap between what generative AI can do in a lab versus what actually holds up in production.

CURRENT RESEARCH

I'm starting a survey paper on how companies are actually using generative AI right now. This isn't about building or training a new model, It's a literature and case-study review. The questions I'm working through:

- How are companies using generative AI in practice, and in which functions?
- Which LLMs see the most real-world adoption, and why?
- What benefits are businesses actually getting from this- cost, speed, customer experience?
- What's still broken or hard: cost, hallucination, governance, integration with legacy systems?
- Where does the literature fall short, and what should researchers look at next?

EDUCATION

Master of Science, Information Technology — Kennesaw State University Aug 2022 – May 2024

Bachelor of Technology, Electronics and Communication Engineering — Geethanjali College of Engineering and Technology Jun 2016 – Nov 2020

ACADEMIC PROJECTS

Discover, Learn, and Protect: A Mobile App for Informal STEM Learning about Local Biodiversity and Environmental Issues

IT Capstone Project | Kennesaw State University | Advisor: Dr. Ying Xie

- Served as **Team Leader and Technical Specialist**, coordinating project planning, task allocation, weekly meetings, and communication with the project owner while supporting team members throughout the project lifecycle.
- Designed and developed an interactive educational web application that delivers personalized learning through quizzes, video lectures, reading assignments, and progress tracking for biodiversity and environmental education.
- Contributed to front-end and back-end development, database design, feature implementation, and integration of application components to deliver a functional web application.
- Integrated **YouTube API** for educational video delivery and **OpenAI ChatGPT API** to provide AI-assisted learning capabilities within the application.
- Participated in application testing, debugging, feature enhancement, and final project presentation following a milestone-driven software development process.

Technologies: HTML, CSS, JavaScript, MongoDB, YouTube API, OpenAI ChatGPT API.

[GitHub](#)

TECHNICAL SKILLS

Programming & Frameworks: Python (PyTorch, TensorFlow, Scikit-learn, XGBoost), LangChain, Hugging Face, JavaScript

AI & Machine Learning: Regression, Classification, Time Series Forecasting, Deep Learning, NLP, RAG, Reinforcement Learning, Generative AI (LLMs)

Data Engineering & Scalability: Apache Spark, Databricks, Snowflake, Kafka, MapReduce, ETL Pipelines, Airflow

MLOps & Deployment: MLflow, Docker, Kubernetes, CI/CD pipelines, SageMaker, Kubeflow

Cloud Platforms: AWS, Azure ML, GCP Vertex AI

Visualization & BI: Tableau, Power BI, Plotly, Seaborn

Mathematics & Statistics: Linear Algebra, Probability, Statistical Inference, Optimization

PROFESSIONAL EXPERIENCE

This is where a lot of my research interest actually comes from, I've spent the last couple years building the kind of systems this paper is about.

Client: USAA — San Antonio, TX Jun 2024 – Present

Role: Data Scientist / AI-ML Engineer

- Deployed production LLM pipelines using GPT-3.5/GPT-4, Anthropic Claude, and Meta LLaMA to power conversational AI, automated document intelligence, and decision-support tools for internal banking teams, good vantage point for seeing how these models actually compare once they're in production, not just on a benchmark.
- Built RAG pipelines on Azure Data Lake using FAISS, Pinecone, and Chroma as vector stores, serving around 500 internal users at roughly 100–200 queries a day for customer-insight summarization and financial-advisory tools grounded in real account and policy data rather than the model's own memory.
- Cut hallucinated or off-base responses by about 25%, driven mainly by retrieval-side changes by chunking strategy, hybrid dense + BM25 search, and re-ranking, rather than prompt tuning alone. That was the clearest lesson from this work: generation quality in a RAG system is bottlenecked by retrieval quality more than by the LLM itself.
- Set up multi-model routing in the same RAG system so it could fall back between GPT-4, Claude, and open-source LLaMA depending on cost, latency, and accuracy for a given query, a small-scale version of the tradeoffs companies make when picking an LLM vendor.
- Built 5 agent workflows in LangChain/LangGraph to orchestrate SDLC-related tasks, modeling them as directed graphs. Python for the deterministic branching/retry logic, the LLM only for the reasoning steps instead of burying all of it in a single prompt.
- Designed an internal MCP (Model Context Protocol) tool-orchestration layer so those agent workflows could call internal REST APIs, databases, and cloud services in a standardized way instead of one-off integrations for each tool. This stayed internal-only, no external MCP connections.
- Used that same agent stack to wire up GitHub Copilot and Cursor for automated code generation and debugging, and built a couple of custom VS Code extensions on top of the LLM APIs for real-time code assistance during development.
- Added drift monitoring and statistical control checks on the deployed forecasting and RAG models, plus explainability output for audit and compliance review, this is where the governance headaches show up in practice, not just in theory.
- Used PyTorch and TensorFlow to build predictive maintenance models that anticipated system failures ahead of time, which pushed operational reliability up by roughly 18%.
- Rebuilt the ETL and inference pipelines feeding the forecasting models across Azure and AWS (Databricks, Snowflake, Kafka, Airflow), cutting data processing latency by around 20%.
- Managed the ML lifecycle end to end data curation through deployment by using MLflow for experiment tracking, Docker/Kubernetes for containerized serving, and automated retraining pipelines to catch data drift before it became a production problem.

Tool Stack: Python (PyTorch, TensorFlow, Scikit-learn, XGBoost), LangChain, Hugging Face, Apache Spark, Databricks, Snowflake, Kafka, Airflow, MLflow, Docker, Kubernetes, AWS SageMaker, Kubeflow, Azure ML, GCP Vertex AI, SQL, Tableau, Power BI, Plotly, Seaborn.

Client: Capita Pvt Ltd — Pune, India Jun 2020 – Jul 2022

Role: Data Scientist

- Used spaCy, NLTK, and BERT to build a document-parsing and sentiment-analysis pipeline over incoming financial compliance documents, flagging risk-relevant language for human review.
- Built demand and sales forecasting models with Prophet and LSTM networks, and separately built classification models for fraud detection, churn prediction, and customer segmentation using scikit-learn.
- Built basic ML API services with Flask and Docker to serve the fraud/forecasting models for inference inside internal clusters .My first real exposure to what it takes to get a model out of a notebook and into something other people can actually call.
- Used Spark and Python to rebuild the ETL workflows feeding those models, mainly to handle large volumes of unstructured text data the older pipeline couldn't process efficiently.
- Contributed interpretable-AI components (feature attribution, decision tracing) on top of the credit-scoring and fraud models, specifically for audit and compliance reviewers who needed to understand why a model flagged something.
- Ran statistical hypothesis testing on the forecasting and classification models to check they were actually stable over time, not just accurate on a single test split.
- Built KPI and model-performance dashboards in Tableau and Power BI for the fraud/forecasting work, and helped set up CI/CD and monitoring for those services on AWS.

Tool Stack: Python, pandas, numpy, scikit-learn, spaCy, NLTK, BERT, Prophet, Spark, SQL, Flask, Docker, Tableau, Power BI, AWS, Jupyter.